Compare this output to the previous example's `mktime()` call with an explicit timezone of UTC.

idate()

The `idate()` function is essentially identical to the `date()` function (which behaves exactly like `new DateTime('@{$timestamp}')->format($format)`) but with the following differences: `idate()` accepts a format string consisting of a single character (be aware that the valid characters for `idate()` are a subset of those supported by the `format` method of `DateTimeInterface`) and an optional integer timestamp and only returns a single integer value (it is not possible to specify multiple units in the formatting string).

```
$tstamp = 1725339600;
var_dump(idate('d', $tstamp));
var_dump(idate('m', $tstamp));
var_dump(idate('y', $tstamp));
var_dump(idate('Y', $tstamp));
```

```
Output:
int(3)
int(9)
int(24)
int(2024)
```

microtime(), hrtime()

The final functions we'll look at in this section are used for high-precision time measurement. `microtime()`, which is fairly ubiquitous in any PHP code concerned with performance, returns the number of microseconds elapsed since the Unix Epoch. `hrtime()` offers nanosecond precision (but from an arbitrary point in time) and was added to the core functions in PHP 7.4 (prior to that it was part of the `HRTime` high-resolution timing extension). The most common use-case for `microtime()` overlaps that of `hrtime()` – namely, for measuring elapsed time between calls – and nanosecond precision is a highly unusual requirement for web applications, so you are far less likely to encounter `hrtime()` in your day-to-day. We'll focus on `microtime()` here, and just be aware that `hrtime()` exists and can be used in similar ways. First let's look at how to call `microtime()`: